Compulsory Assignment 1: Dense neural networks - Implementing an ANN with Keras

Please fill out the the group name, number, members and optionally the group name below.

Group number: 3
Group member 1: Sara Marie Åseng Almdahl
Group member 2: Fredrik Gard Haugen
Group member 3: Kim Andre Nielsen
Group name (optional):

Assignment submission

To complete this assignment, answer the all the questions in this notebook and write the code required to implement different models. **Submit the assignment by handing in this notebook as both an .ipynb file and a .pdf file**.

Here are some do's and don'ts for the submission:

- Read questions thoroughly before answering.
- Make sure to answer all questions.
- Ensure all code cells are run.
- Label all axes in plots.
- Ensure all figures are visible in the PDF.
- Provide a brief explanation of how your code works.

Introduction

In this assignment we will work with the task of classifying hand gestures from the Sign Language MNIST dataset. This time you will implement the network using the Keras API of the TensorFlow library. TensorFlow and PyTorch are both free open-source software libraries intended to simplify multiplication of tensors, but are mostly used for the design and implementation of deep neural networks. Both libraries simplify the implementation of neural networks, and allow for faster training of networks by utlizing hardware acceleration with Graphical Processing Units (GPUs) or Tensor Processing Units (TPUs)

TensorFlow was developed by Google Brain for internal use in Google and was initially released under Apache 2.0 License in 2015 [1]. Keras was initially released as separate software library, developed by François Chollet, to simplify the Python interface for design of artificial neural networks. Up until version 2.3 Keras supported multiple backend libraries including TensorFlow, Microsoft Cognitive Toolkit, Theano, and PlaidML

2. When TensorFlow 2.0 was released in 2019, keras was included as a TensorFlow specific API that is accessible by:

```
import tensorflow.keras as ks
```

PyTorch was originally developed by Meta AI (formerly known as Facebook) in 2016, but is now under umbrella of the Linux foundation, and is open-source under the BSD license 3. While TensorFlow was the most popular framework for a long time, PyTorch has been gaining more and more users in the last five years and is now more used in industry and is becoming more popular in research as well.

The lectures of DAT300 will be taught using the Keras API in TensorFlow, and we recommend you to stick with Keras and TensorFlow for this course as it is easier for beginners to get started with.

Dataset descirption

The Sign Language MNIST dataset is a collection of grayscale images of size 28×28 pixels, representing hand gestures for the letters A–Y in American Sign Language (ASL). The letters J and Z are excluded since they involve motion.

- Training set: 27,455 images
- Test set: 7,172 images
- Number of classes: 24 (letters A–Y, excluding J and Z)
- Format: Each image is stored as a flattened vector of 784 pixels, with an accompanying label indicating the class (0–23).

This dataset is commonly used as a benchmark for image classification tasks, similar to the original MNIST handwritten digits dataset, but adapted for sign language recognition.

Assignment structure

1. Part 1: Import, preprocess, and visualize the data.
2. Part 2: Design your own Dense Neural Network (NN) architecture for classifying MNIST in Keras.
3. Part 3: Train one of the Machine Learning classifiers that you learned about in DAT200.
4. Part 4: Compare and discuss the results.

Supporting code

You may find the code below useful for plotting the metrics and calculating the F1-score for your model in Part 2.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import f1_score
import tensorflow as tf

"""
```

```
A function that plots the training and validation metrics over epochs
"""
def plot_training_history(training_history_object, list_of_metrics=None):

    """
    training_history_object: Object returned by model.fit() function in keras
    list_of_metrics: A list of MAX two metrics to be plotted
    """
    history_dict = training_history_object.history
    if list_of_metrics is None:
        list_of_metrics = [key for key in list(history_dict.keys()) if 'val_' r

    trainHistDF = pd.DataFrame(history_dict)
    train_keys = list_of_metrics
    valid_keys = ['val_' + key for key in train_keys]
    nr_plots = len(train_keys)
    fig, ax = plt.subplots(1,nr_plots,figsize=(5*nr_plots,4))
    for i in range(len(train_keys)):
        ax[i].plot(np.array(trainHistDF[train_keys[i]]), label='Training')
        ax[i].plot(np.array(trainHistDF[valid_keys[i]]), label='Validation')
        ax[i].set_xlabel('Epoch')
        ax[i].set_title(train_keys[i])
        ax[i].grid('on')
        ax[i].legend()
    fig.tight_layout
    plt.show()

"""
Custom Keras callback for computing the F1-score after each epoch.
This callback calculates both training and validation F1-scores
"""
class F1ScoreCallback(tf.keras.callbacks.Callback):
    def __init__(self, X_train, y_train, X_val, y_val):
        super().__init__()
        self.X_train, self.y_train = X_train, y_train
        self.X_val, self.y_val = X_val, y_val

    def on_epoch_end(self, epoch, logs=None):
        # Training F1
        y_train_pred = self.model.predict(self.X_train, verbose=0).argmax(axis=
        f1_train = f1_score(self.y_train, y_train_pred, average="macro")

        # Validation F1
        y_val_pred = self.model.predict(self.X_val, verbose=0).argmax(axis=1)
        f1_val = f1_score(self.y_val, y_val_pred, average="macro")

        logs["f1"] = f1_train
        logs["val_f1"] = f1_val
        print(f" — f1: {f1_train:.4f} — val_f1: {f1_val:.4f}")
```

Library imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import f1_score
import tensorflow as tf
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import seaborn as sns
import time
```

Task 1: Importing, preprocess and visualizing the data

In this assignment you yourselves will be responsible for the data-preprocessing. Use the cells below for preprocessing and visualization, and optionally some exploration of the dataset if you feel inclined.

Importing data

You will need to upload both CSV files from the zip folder to Google Colab and run the code sell below to load the data.

```
# Sign language MNIST
train = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/DAT300 gruppe3/CA1/
test  = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/DAT300 gruppe3/CA1/

X_train = train.drop(columns=['label']).to_numpy(dtype=np.float32)
y_train = train['label'].to_numpy(dtype=np.int32)
X_test  = test.drop(columns=['label']).to_numpy(dtype=np.float32)
y_test  = test['label'].to_numpy(dtype=np.int32)
```

Task 1.1 Preprocessing

Preprocess the data in whatever way you find sensible. Remember to comment on what you do.

```
# Scaling the data
scaler = StandardScaler()
scaler.fit(X_train)
scaler.transform(X_train)
scaler.transform(X_test)
```

```
array([[ 0.08657672,  0.01251149, -0.03194715, ..., -0.7893898 ,
        -0.64672816, -0.8203149 ],
       [-0.46954566, -0.5132584 , -0.5184334 , ...,  0.34809467,
         0.32797274,  0.31330162],
       [-1.4608942 , -1.514725  , -1.5170105 , ...,  0.99582887,
         0.988254  ,  0.96551937],
```

```
       ...,
       [ 1.0779253 ,  1.0640513 ,  0.9922344 , ...,  0.77465135,
         0.75243926,  0.74811345],
       [ 1.343897  ,  1.4145646 ,  1.453116  , ..., -1.5003176 ,
        -1.4327773 , -1.5035906 ],
       [ 0.66687834,  0.6384281 ,  0.55695724, ...,  0.52187705,
         0.50090355,  0.49964955]], dtype=float32)
```

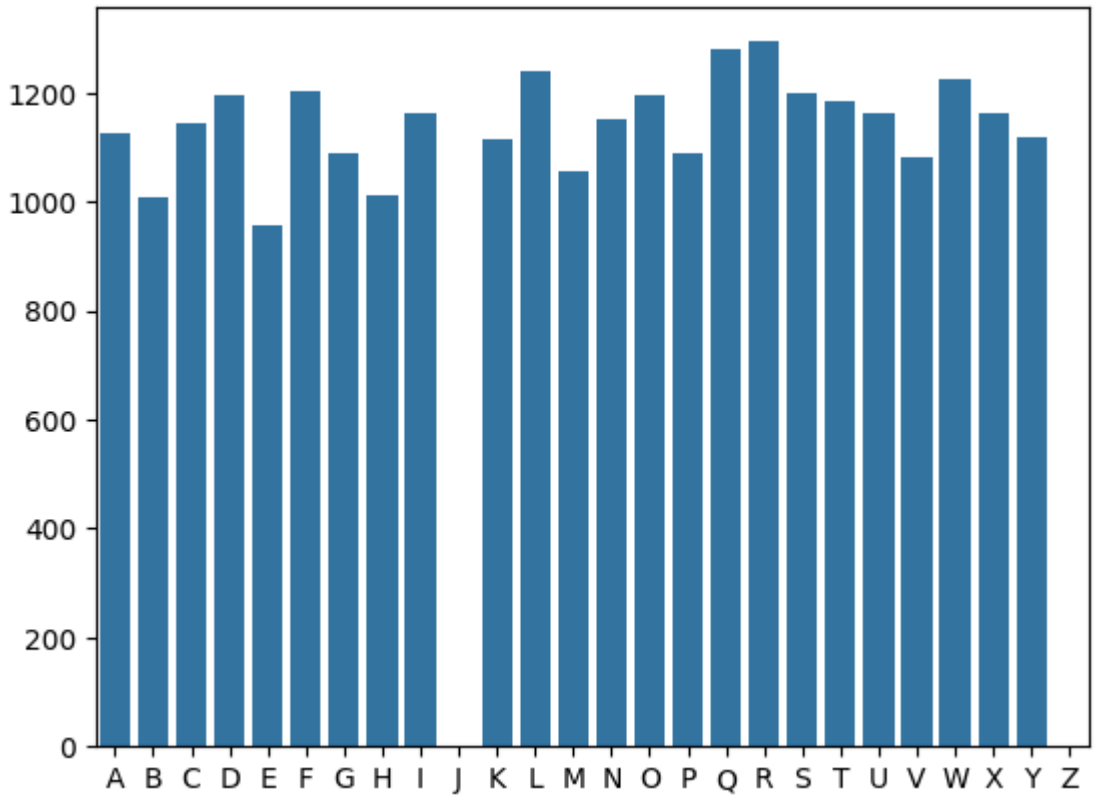
Task 1.2 Visualization

Visualize the data in whatever manner you find helpful/sensible and briefly comment on the plots.

```
# See what we have of letters
letters = np.array(['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L',
amountLetters = np.zeros(shape=(26), dtype="int32")
for val in y_train:
  amountLetters[val] += 1

sns.barplot(x=letters, y=amountLetters)
print("X_train shape:", np.shape(X_train))
print("y_train shape:", np.shape(y_train))
```

```
X_train shape: (27455, 784)
y_train shape: (27455,)
```



We have a 27455 of examples of the relevant letters (not J & Z). The assigment stated that the the classes were 0-23 but they seem to be 0-25, with no values at 9 (J) and 25 (Z). We therfore switch the target values so we have classes 0-23.

```
# Run this block once

print("Number of targets at index 9 (J), in the training data, before restackir
print("Number of targets at index 10 (K) in the training data, before restackir
y_train[y_train > 9] -= 1
print("\nNumber of targets at index 9 (K), in the training data, after restacki

print("\nNumber of targets at index 9 (J), in the test data, before restacking:
print("Number of targets at index 10 (K) in the test data, before restacking:",
y_test[y_test > 9] -= 1
print("\nNumber of targets at index 9 (K), in the test data, after restacking:"

# Delete letters J and Z from our letter array
np.delete(letters, 9)
np.delete(letters, 25);


#y_train_oneHot = one_hot_encoder(y_train, maxVal=23)
#y_test_oneHot = one_hot_encoder(y_test, maxVal=23)
```

```
Number of targets at index 9 (J), in the training data, before restacking: 0
Number of targets at index 10 (K) in the training data, before restacking: 1114

Number of targets at index 9 (K), in the training data, after restacking: 1114

Number of targets at index 9 (J), in the test data, before restacking: 0
Number of targets at index 10 (K) in the test data, before restacking: 331

Number of targets at index 9 (K), in the test data, after restacking: 331
```

Dobbeltklikk (eller trykk på Enter) for å redigere

```
Begynn å kode, eller generer med AI.
```

Task 2: Design your own ANN architecture

In this task you are free to design the network architecture for the MNIST Gesture recognition challenge with a couple of stipulations:

- use **only Dense or fully connected layers**,
- use both **accuracy and the F1-score** as performance metrics.

Otherwise, you are free to use whatever loss-function, optimizer and activation functions you want and train it for as many epochs you want.

Task 2.1: Implement your own network architecture

Design your network below:

(Feel free to add as many code and markdown cells as you want)

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Flatten(),
    layers.Dense(128, activation="sigmoid", activity_regularizer=keras.regulari
    layers.Dense(64, activation="sigmoid", activity_regularizer=keras.regulariz
    layers.Dense(24, activation="softmax", activity_regularizer=keras.regulariz
])

model.summary()

optimizer = keras.optimizers.SGD(learning_rate=0.0075)

# Compile model
model.compile(optimizer=optimizer,
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy']
              )
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
flatten_5 (Flatten)	(None, 784)	0
dense_15 (Dense)	(None, 128)	100,480
dense_16 (Dense)	(None, 64)	8,256
dense_17 (Dense)	(None, 24)	1,560

Total params: 110,296 (430.84 KB)
Trainable params: 110,296 (430.84 KB)
Non-trainable params: 0 (0.00 B)

Task 2.2: Train your network and visualize the training history

Train the model and plot the training history in the code cell(s) below. Feel free to use the function `plot_training_history()` and the custom Keras callback `F1ScoreCallback()` from Supporting code section.

```
X_train_sub, X_val, y_train_sub, y_val = train_test_split(X_train, y_train, tes

startTime_ann = time.perf_counter()

training_history_object = model.fit(X_train_sub,
          y_train_sub,
```

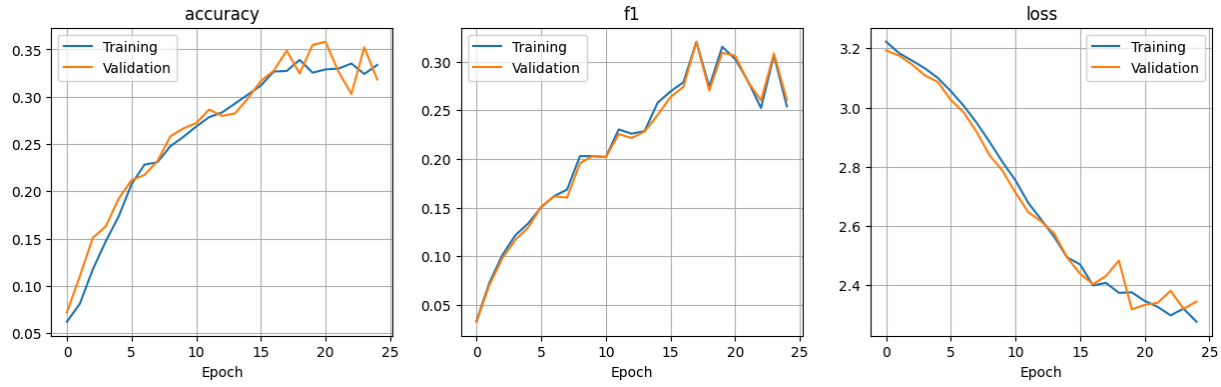
```
            validation_data=(X_val, y_val),
            epochs=25,
            batch_size=50,
            callbacks = [F1ScoreCallback(X_train_sub, y_train_sub, X_val, y_val)]

endTime_ann = time.perf_counter()
```

```
Epoch 7/25
439/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.2250 - loss: 3.0170 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.2251 - loss: 3.0165 - v
Epoch 8/25
448/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.2217 - loss: 2.9706 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.2219 - loss: 2.9701 - v
Epoch 9/25
434/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.2435 - loss: 2.8980 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.2437 - loss: 2.8972 -
Epoch 10/25
433/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.2509 - loss: 2.8367 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.2512 - loss: 2.8356 - v
Epoch 11/25
433/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.2639 - loss: 2.7679 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.2641 - loss: 2.7672 - v
Epoch 12/25
436/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.2809 - loss: 2.6925 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.2808 - loss: 2.6918 - v
Epoch 13/25
454/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.2843 - loss: 2.6303 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.2843 - loss: 2.6302 - v
Epoch 14/25
444/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.2898 - loss: 2.5864 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.2898 - loss: 2.5856 - v
Epoch 15/25
438/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.2984 - loss: 2.5061 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.2985 - loss: 2.5055 - v
Epoch 16/25
453/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.3117 - loss: 2.4776 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - accuracy: 0.3117 - loss: 2.4775 - v
Epoch 17/25
441/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.3299 - loss: 2.4047 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - accuracy: 0.3298 - loss: 2.4045 - v
Epoch 18/25
```

```
441/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.3242 - loss: 2.3319 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.3242 - loss: 2.3315 - '
Epoch 25/25
444/458 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.3372 - loss: 2.2840 – f
458/458 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.3371 - loss: 2.2838 - v
```

```
plot_training_history(training_history_object, ['accuracy', 'f1', 'loss'])
```



Task 2.3: Discuss the results

Question 2.3.1: What could happen if the model is too shallow or too deep?

Question 2.3.2: How does the choice of train/validation/test split ratio affect the training process and the final performance of a deep learning model?

Question 2.3.3: How do accuracy and F1-Score values compare (are they similar or very different from each other)? What does it tell you about the MNIST dataset and which one of these metrics is more reliable in this case?

Question 2.3.4: Explain **very briefly** how each of the following model hyperparameters can impact the model's performance:

- Number of layers
- Number of neurons in a layer
- Batch size
- Optimizers
- Regularization techniques (such as L2 regularization)

Dobbeltklikk (eller trykk på Enter) for å redigere

2.3.1) If the model is to shallow it won't pick up on all the importent/promenent patterns in the data (underfitted). If the model is to deep it may become sencitive to noize (overfitted).

2.3.2) If the split results in few test samples than the model may be too shallow. If the validation set is too small, then the tuning of the model may be unrepresented of the dataset. If the test set is too small, then the evaluation of the model may be wrong.

2.3.3) The f1-score seems to be about 5% points under the accuracy. Since f1 ≈ accuracy, this implies that the dataset is well distributed

2.3.4)

- A large number of layers may lead to a vanashing gradient. The number of layeres also determin if the model is too shallow or too deep
- Too many nuerons may lead to overfitting and too few may lead to underfitting
- Smaller batches improve generalization, but slow down the treining. Larger batches speed up training but may reduce model robustness
- Different optimizer optimize differently.
- Regularization techniques are used to shrink the weigths so we don't get enormes weigths. L1 can do feature extraction as well as general regularization and L2 does general regularization

Task 3: Design and train a classical machine learning classifier

Pick your **favourite** machine learning classifer that you learned about in DAT200 and train it for the MNIST gesture recognition problem. (Hint: use the scikit-learn library). Remember to use **accuracy and the F1-score** as performance metrics.

Imports

```
import pandas as pd
import numpy as np
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.impute import SimpleImputer
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score,
from sklearn.svm import SVC, LinearSVC
from scipy.stats import loguniform
from pathlib import Path
```

Make pipeline for polynomial SVC

```
pipe_psvc = make_pipeline(
    SimpleImputer(missing_values=np.nan, strategy="mean"),
    StandardScaler(),
    PCA(n_components=10),
    SVC(kernel="poly", random_state=42, max_iter=50)
)
```

Do randomised search for optimizing hyperparameters

```
param_dist = {
    'svc__gamma': loguniform(1e-3, 1e2),
    'svc__C': loguniform(1e-3, 1e2),
    'svc__degree': [1,2,3]
}

inner_segments = 3
outer_segments = 3

search_psvc = RandomizedSearchCV(
    pipe_psvc,
    param_distributions=param_dist,
    cv=inner_segments,
    n_iter=5,
    verbose=3,
    scoring="accuracy"
)

psvc_scores = cross_val_score(search_psvc, X_train, y_train,
                          scoring='accuracy', cv=outer_segments)


search_psvc = RandomizedSearchCV(
    pipe_psvc,
    param_distributions=param_dist,
    cv=2,
    n_iter=1,
    verbose=1,
    scoring="accuracy",
    random_state=42
)


startTime_cla = time.perf_counter()
psvc_scores = cross_val_score(search_psvc, X_train, y_train,
                          scoring='accuracy', cv=2)
```

```
print('CV accuracy: %.3f +/- %.3f' % (np.mean(psvc_scores),
                                      np.std(psvc_scores)))
search_psvc.fit(X_train,y_train)


print("Best score:", search_psvc.best_score_)
print("Best parameters", search_psvc.best_params_)
```

```
Fitting 2 folds for each of 1 candidates, totalling 2 fits
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:305: ConvergenceW
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:305: ConvergenceW
  warnings.warn(
Best score: 0.2701510228043247
Best parameters {'svc__C': np.float64(0.0745934328572655), 'svc__degree': 1, '
/usr/local/lib/python3.12/dist-packages/sklearn/svm/_base.py:305: ConvergenceW
  warnings.warn(
```

```
ev_psvc = make_pipeline(
    SimpleImputer(missing_values=np.nan, strategy="mean"),
    StandardScaler(),
    PCA(n_components=10),
    SVC(kernel="poly", gamma=0.008263688714158009, C=0.0745934328572655, degree
)



ev_psvc.fit(X_train, y_train)
endTime_cla = time.perf_counter()
```

Task 4: Compare and discuss

Evaluate the ANN model you implemented in Task 2 against the classical machine learning model from Task 3, using the test dataset. Compare the two models based on:

- Accuracies and F1-scores they attain
- Time it takes to train them

Did you experience any trouble when training models in tasks 2 and 3?

```
# Make predictions
y_pred_cla = ev_psvc.predict(X_test)

print('Classical machine learning (SVC):')

# Calculate f1
f1_cla = f1_score(y_test, y_pred_cla, average='macro')
print(f'- Classification f1: {f1_cla * 100:.2f}%')

# Calculate accuracy
accuracy_cla = accuracy_score(y_test, y_pred_cla)
print(f'- Classification Accuracy: {accuracy_cla * 100:.2f}%')

# Calculate timeing
print(f'- Training time: {endTime_cla - startTime_cla:.2f}s')
```

```
Classical machine learning (SVC):
- Classification f1: 46.33%
- Classification Accuracy: 49.44%
```

```
# Make predictions
y_pred_ann = model.predict(X_test).argmax(axis=1)

print('Artificial neural network:')

# Calculate f1
f1_ann = f1_score(y_test, y_pred_ann, average='macro')
print(f'- Classification f1: {f1_ann * 100:.2f}%')

# Calculate accuracy
accuracy_ann = accuracy_score(y_test, y_pred_ann)
print(f'- Classification Accuracy: {accuracy_ann * 100:.2f}%')

# Calculate timeing
print(f'- Training time: {endTime_ann - startTime_ann:.2f}s')
```

```
225/225 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step
Artificial neural network:
- Classification f1: 24.91%
- Classification Accuracy: 28.61%
- Training time: 88.09s
```

Begynn å kode, eller generer med AI.

Task 4 discussion Here:

From the results we can see that the classical machine learning model, a polynomial SVC model in our case, performed better than the ANN model. This may be due to the SVC model having been more finetuned with random search for optimizing parameters, which we have thus far not learnt how to do for ANN models. But, our results atleast establish that a SVC model with optimised hyperparameters can outperform an ANN model without finetuned hyperparameters. The training time is similiar